Hyper-V vs. VMware: Overhead da Virtualização

Henrique Laia (M16491)
Departamento de Informática
Universidade da Beira Interior
Covilhã, Portugal
m16491@ubi.pt

Alexandre Santos (**n.º**)
Departamento de Informática
Universidade da Beira Interior
Covilhã, Portugal
número@ubi.pt

Daniel Basílio (**n.º**)
Departamento de Informática
Universidade da Beira Interior
Covilhã, Portugal
número@ubi.pt

***Resumo*—Este artigo compara um hipervisor nativo (Hyper-V) com um hipervisor baseado em anfitrião (VMware Workstation 17.6) no mesmo portátil, com convidados Ubuntu Server 26.04 idênticos e uma referência sem virtualização ($N = 10$, teste t de Welch). Face ao nativo, o processador perde 2–11% nos dois hipervisores, e o acesso aleatório à memória 27% no Tipo 1 e 47% no Tipo 2. No armazenamento, o Tipo 2 é mais rápido com fila unitária, graças à *cache* do anfitrião, mas perde 88% de IOPS com 32 pedidos em fila, uma saturação que a lei de Little explica. Na rede, perde 33–66% de débito.**

***Palavras-chave*—virtualização, Hyper-V, VMware Workstation, hipervisor Tipo 1, hipervisor Tipo 2, overhead, E/S**

I. Introdução

A virtualização é a base dos centros de dados e da computação em nuvem: permite consolidar vários sistemas isolados num único servidor físico, melhorando a utilização do hardware e simplificando a gestão [3], [4]. O componente que o torna possível, o hipervisor, interpõe-se entre o hardware e os sistemas convidados e decide como cada instrução privilegiada, interrupção ou pedido de Entrada/Saída (E/S) chega ao hardware. A sua arquitetura condiciona, por isso, o desempenho e a previsibilidade dos serviços alojados.

Na taxonomia clássica [4], [7], um hipervisor de Tipo 1 (nativo ou *bare-metal*) executa diretamente sobre o hardware, enquanto um de Tipo 2 (*host-based*) executa como aplicação sobre um sistema operativo anfitrião. É comum assumir-se que o Tipo 2 é sempre mais lento. Contudo, as extensões de virtualização assistida por hardware tornaram quase nativo o custo de executar código [9], [10], e o Tipo 2 pode beneficiar de mecanismos do anfitrião, como a *cache* de ficheiros. A questão relevante é, assim, *onde* está o *overhead* e em que condições se manifesta.

Este trabalho instala e configura, em *single node*, o Microsoft Hyper-V [1] e o VMware Workstation [2] no mesmo equipamento, e quantifica o *overhead* relativo nos quatro subsistemas: processador, memória, armazenamento e rede. As contribuições são: (i) uma comparação controlada, com recursos e topologia de vCPU equivalentes, verificação explícita do modo de execução de cada hipervisor e análise de significância estatística; (ii) a identificação de dois regimes opostos no armazenamento, explicados pela lei de Little [14]; (iii) a quantificação do *overhead* de cada hipervisor face ao sistema nativo; e (iv) uma experiência que avalia o efeito do escalonador do Hyper-V.

O artigo organiza-se da seguinte forma: a Secção II apresenta o enquadramento e os trabalhos relacionados; a Secção III descreve a arquitetura e os métodos; a Secção IV, a configuração experimental; a Secção V apresenta e discute os resultados; a Secção VI identifica as limitações; e a Secção VII conclui.

Trabalho realizado no âmbito da Unidade Curricular de Virtualização e Centros de Dados, Departamento de Informática, Universidade da Beira Interior.

II. Background e Trabalhos Relacionados

A. Virtualização x86 Assistida por Hardware

Popek e Goldberg [7] estabeleceram que uma arquitetura é virtualizável se todas as instruções sensíveis forem privilegiadas, condição que o x86 original não cumpria. Isto levou primeiro à tradução binária [9] e à paravirtualização, como no Xen [8]. As extensões Intel VT-x e AMD-V acrescentaram um modo de execução para o hipervisor, e as tabelas de páginas de segundo nível (EPT/NPT) passaram a traduzir em hardware os endereços do convidado [10]. Esta tradução tem, porém, um custo: uma falha de TLB obriga a percorrer as tabelas do convidado e as do hipervisor, um percurso bidimensional que pode exigir até 24 acessos à memória com páginas de 4 KiB e que é muito mais curto quando o hipervisor usa páginas grandes (2 MiB) [10]. O restante custo concentra-se nas saídas para o hipervisor (*VM exits*), sobretudo no acesso a dispositivos.

Os dispositivos virtuais podem ser *emulados*, imitando hardware real (por exemplo, a placa de rede Intel `e1000` ou o controlador SCSI LSI Logic), o que obriga a uma saída para o hipervisor em cada acesso a registos; ou *paravirtualizados*, com controladores que sabem estar virtualizados e trocam pedidos em lote por memória partilhada [8], [9].

B. Hipervisor Nativo: Microsoft Hyper-V

No Hyper-V, o hipervisor (`hvix64`/`hvax64`) é carregado no arranque, antes do Windows, que passa a executar como *partição raiz*. As máquinas convidadas executam em *partições filhas* e usam dispositivos paravirtualizados: os controladores do convidado (VSC, como `hv_storvsc` e `hv_netvsc`) enviam pedidos pelo barramento *VMBus* aos fornecedores de serviço (VSP) da partição raiz, em modo núcleo [1]. Tanto no Windows Server como no Windows 11, o Hyper-V é um hipervisor de Tipo 1. As edições diferem nas funcionalidades

de centro de dados (*Failover Clustering*, Hyper-V Replica) e no escalonador predefinido: *root* nas edições cliente, *classic* no Windows Server 2016 e *core* a partir do 2019 [11]. No *root scheduler*, os vCPU são escalonados pelo próprio Windows como *threads*. No *core scheduler*, o hipervisor atribui núcleos físicos inteiros a cada máquina, nunca partilhando os *threads* SMT de um núcleo entre máquinas diferentes, por isolamento de segurança [11].

C. Hipervisor Baseado em Anfitrião: VMware Workstation

O VMware Workstation instala um monitor (VMM) e um controlador no núcleo do Windows, mas a emulação de dispositivos e o acesso ao ficheiro de disco virtual (`.vmdk`) são feitos pelo processo `vmware-vmx`, em modo utilizador [2], [9]. Assim, a E/S do convidado chega ao NTFS através da API do Windows e pode ser servida pela *cache* de ficheiros do anfitrião. A memória do convidado é, ela própria, memória do processo `vmware-vmx`, gerida pelo Windows. Quando o hipervisor do Windows está ativo (funcionalidade Hyper-V, segurança baseada em virtualização ou WSL2), o VMware não pode usar o seu monitor nativo (modo `CPL0`) e executa sobre a *Windows Hypervisor Platform* (modo `ULM`) [12]: nesse caso, já não se trata de um Tipo 2 "puro".

D. Trabalhos Relacionados

Adams e Agesen [9] mostraram que as primeiras gerações de VT-x eram, em certas cargas, mais lentas do que a tradução binária, situação revertida com EPT/NPT [10]. Barham *et al.* [8] quantificaram o custo da paravirtualização no Xen, e Felter *et al.* [13] reportaram *overheads* de processador e memória desprezáveis em KVM, com a penalização concentrada na E/S. Na Universidade da Beira Interior, Louro [3] comparou uma infraestrutura de elevada disponibilidade com Hyper-V e outra com contentores Docker, usando o IOzone (sistema de ficheiros), o RAMspeed (memória), o iPerf3 (rede) e o Geekbench (processador), em três ambientes por infraestrutura (nativo, virtualizado e aninhado), sem indicar o número de repetições nem a dispersão dos resultados. Este trabalho avalia os mesmos subsistemas, mas com ferramentas abertas e parametrizáveis, cujos resultados se exprimem em unidades físicas: `sysbench` [5] para o processador e a memória, `fio` [6] para o disco, com E/S direta e várias profundidades de fila, e `iperf3` para a rede. Acrescenta ainda repetições e análise estatística.

III. Arquitetura e Métodos

A. Arquitetura dos Cenários

A Fig. 1 mostra as duas pilhas ensaiadas. No Tipo 1, os pedidos de disco e de rede do convidado seguem por controladores paravirtualizados até à partição raiz, tratados em modo núcleo. No Tipo 2, chegam a dispositivos emulados (LSI Logic e `e1000`, a configuração predefinida da máquina) que o processo `vmware-vmx` traduz em operações sobre ficheiros e sobre o adaptador virtual `VMnet8` do anfitrião.

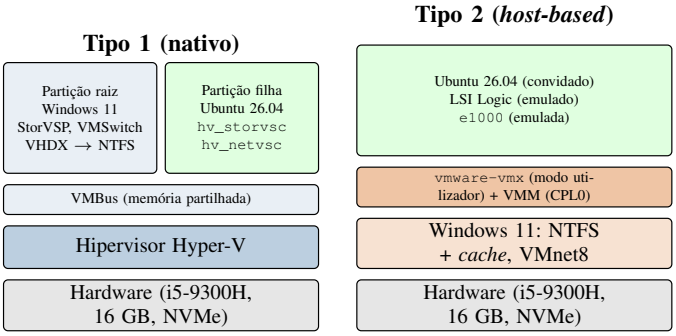


Figura 1. Pilhas de software dos dois cenários e respetivos dispositivos virtuais.

B. Formulação Estatística

Cada métrica foi medida em $N = 10$ repetições. Calcula-se a média μ, o desvio-padrão amostral corrigido

$$\sigma = \sqrt{\frac{1}{N-1}\sum_{i=1}^{N}(x_i - \mu)^2}, \tag{1}$$

e o intervalo de confiança a 95%, $\mu \pm t_{0{,}975;9}\,\sigma/\sqrt{N}$, com $t_{0{,}975;9} = 2{,}262$. As diferenças entre cenários são avaliadas com o teste t de Welch (variâncias desiguais), considerando-se significativas para $p < 0{,}05$.

C. Quantificação do Overhead

Com o Hyper-V como referência, a variação relativa do Tipo 2 é

$$\Delta\% = \frac{\mu_{\text{Tipo 2}} - \mu_{\text{Tipo 1}}}{\mu_{\text{Tipo 1}}} \times 100. \tag{2}$$

Em métricas de débito (eps, MiB/s, IOPS, Mbit/s), $\Delta\% < 0$ indica pior desempenho do Tipo 2; em métricas de latência, $\Delta\% > 0$ indica pior desempenho.

D. Modelo de Filas para o Armazenamento

Pela lei de Little [14], num sistema em regime estacionário com Q pedidos em curso e latência média $\bar{L}$, o débito é

$$\text{IOPS} = \frac{Q}{\bar{L}}. \tag{3}$$

Se um caminho de E/S consegue servir pedidos em paralelo, aumentar Q aumenta os IOPS com pouco acréscimo de $\bar{L}$. Se o caminho estiver saturado, os IOPS estabilizam e $\bar{L}$ cresce proporcionalmente a Q. Este modelo permite distinguir entre um caminho *lento* e um caminho *serializado*.

IV. Configuração Experimental

A. Sistema Anfitrião e Máquinas Virtuais

Os dois cenários foram executados no mesmo portátil (Tabela I), com Windows 11 Home 25H2 (*build* 26200). A Tabela II resume as máquinas virtuais: recursos iguais e o mesmo sistema convidado, com o *kernel* e as versões das ferramentas confirmados como idênticos nos dois cenários.

Tabela I
SISTEMA ANFITRIÃO

Componente	Especificação
Processador	Intel Core i5-9300H, 4 núcleos / 8 *threads*, 2,4 GHz
Memória	16 GB DDR4-2666 (2 × 8 GB)
Armazenamento	SSD NVMe Samsung PM981 (MZVLB1T0), 1 TB
Sistema operativo	Windows 11 Home 25H2, *build* 26200
Energia	Plano "Elevado desempenho", ligado à corrente

Tabela II
CONFIGURAÇÃO DAS MÁQUINAS VIRTUAIS

Parâmetro	Hyper-V (Tipo 1)	VMware (Tipo 2)
Versão	config. VM 12.0, Geração 2	Workstation 17.6.4, HW v21
Convidado	Ubuntu Server 26.04.1 LTS, *kernel* 7.0.0-38	
vCPU	2, sem SMT (1 *socket* × 2)	2, sem SMT (2 *sockets* × 1)
Memória	4096 MB (fixa)	4096 MB
Disco (25 GB)	`.vhdx` dinâmico	`.vmdk` esparso
Controlador de disco	SCSI sintético (VMBus)	LSI Logic (emulado)
Escalonador E/S conv.	`none`	`mq-deadline`
Rede	`hv_netvsc`, *Default Switch*	`e1000`, NAT (VMnet8)
Sistema de ficheiros	`ext4` sobre LVM	
Ferramentas	`sysbench` 1.0.20, `fio` 3.41, `iperf3` 3.20	
Servidor de rede	`iperf3` 3.22 no Windows anfitrião	

B. Instalação e Protocolo de Isolamento

O Hyper-V foi ativado através das funcionalidades opcionais do Windows (`Microsoft-Hyper-V-All`). Como esta funcionalidade não está disponível de origem na edição Home, foi instalada **[descrever o método usado]**. A máquina foi criada no Gestor Hyper-V como Geração 2, com memória dinâmica desativada. Antes dos ensaios, o *checkpoint* automático foi fundido, para que o disco não fosse um VHDX de diferenciação. O VMware Workstation foi instalado a partir do instalador oficial, e a máquina foi criada com a configuração predefinida para Ubuntu de 64 bits. Em ambos os casos, o Ubuntu Server 26.04 foi instalado com as opções predefinidas.

Por omissão, o Hyper-V apresenta ao convidado os dois vCPU como dois *threads* SMT de um único núcleo, enquanto o VMware os apresenta como processadores independentes. Para que a comparação incida apenas no hipervisor, desativou-se a exposição de SMT no Hyper-V (`Set-VMProcessor -HwThreadCountPerCore 1`). Assim, ambos os convidados dispõem de dois vCPU independentes, sem SMT, num único nó NUMA, como se confirmou com `lscpu`. A única diferença que resta é a forma como os vCPU são agrupados: num *socket* no Hyper-V e em dois no VMware. Como o hipervisor do Windows não permite definir *sockets* sem criar nós NUMA virtuais, esta forma foi mantida. Não tem efeito no desempenho, pois o anfitrião tem um único processador físico.

Cada cenário exigiu um modo de arranque distinto do anfitrião, confirmado após cada reinício:

- **Hyper-V:** hipervisor ativo (`hypervisorlaunchtype auto`), com escalonador *root* (tipo `0x4`, lido no registo de eventos do Windows).
- **VMware:** hipervisor e Integridade de Memória desativados (`hypervisorlaunchtype off`), com `HypervisorPresent = False` e o registo `vmware.log` a indicar `Monitor Mode: CPL0`.

Esta verificação é indispensável: com o hipervisor do Windows ativo, o VMware executaria em modo `ULM`, sobre o Hyper-V, e deixaria de representar um Tipo 2.

Em ambos os cenários: apenas uma máquina virtual em execução; aplicações do anfitrião encerradas (incluindo a sincronização do OneDrive) e Windows Update em pausa; pastas das máquinas excluídas da análise em tempo real do Microsoft Defender; plano de energia "Elevado desempenho" com o equipamento sempre ligado à corrente. Antes de cada bateria, aguardaram-se 10 minutos após o arranque, e as tarefas automáticas de atualização do convidado (`apt`) foram suspensas. Os testes foram lançados por SSH dentro de uma sessão `tmux` no convidado.

C. Cargas de Trabalho

As cargas foram automatizadas num *script* único e executadas de forma idêntica nos dois convidados (Tabela III). Cada bateria inclui uma corrida de aquecimento descartada, seguida de $N = 10$ repetições *intercaladas*: em cada repetição alternam processador, memória e disco, para que perturbações lentas do anfitrião não afetem apenas um teste. Há pausas de 5 s entre testes, e a *cache* de páginas do convidado é limpa antes de cada teste de disco (`drop_caches`). Os testes de rede usaram um servidor `iperf3` no Windows anfitrião, com 10 repetições e aquecimento. Como a configuração é de nó único (*single node*), o tráfego circula entre o convidado e o anfitrião apenas através do comutador virtual de cada hipervisor, sem passar por uma placa de rede física. Mede-se, assim, exclusivamente o custo da virtualização da rede, que de outro modo ficaria mascarado pelo limite de uma ligação física (cerca de 1 Gbit/s), igual para os dois cenários. Os comandos exatos, os *scripts* e os dados brutos estão disponíveis no repositório do trabalho [15], para replicação.

Sistema de referência. Para quantificar o *overhead* face ao sistema sem virtualização, os testes foram repetidos no mesmo equipamento, sem nenhuma máquina virtual, com a mesma ferramenta e os mesmos parâmetros de cada subsistema. No processador e na memória, o `sysbench` foi executado num Ubuntu 26.04.1 nativo, arrancado a partir de uma *pen* USB (*live*, *kernel* 7.0.0-30), com o perfil de energia "desempenho". Os dois *threads* foram fixados em dois núcleos físicos distintos (`taskset`), como os dois vCPU sem SMT das máquinas virtuais. O *turbo* foi desligado (`intel_pstate/no_turbo`), limitando o processador a 2,4 GHz: medições no Windows anfitrião mostraram que este executa sempre a cerca de 2,35 GHz, sem *turbo*, em todos os planos de energia, pelo que é essa a frequência a que correram as máquinas virtuais. Com o *turbo* ligado, o nativo atinge cerca de 4 GHz e é 67–71% mais rápido no processador, o que mediria a frequência e não a virtualização. No disco e na rede, a referência foi o Windows anfitrião, com o hipervisor desligado (o modo do cenário VMware): o `fio` 3.41 com os seis perfis da Tabela III (`windowsaio`, `direct=1`, ficheiro de 4 GiB) e o `iperf3` 3.22 em *loopback* (127.0.0.1), como limite superior

Tabela III
CARGAS DE TRABALHO (POR REPETIÇÃO)

Subsistema	Ferramenta e parâmetros	Duração
CPU	`sysbench cpu`, primos até 20 000; 1 e 2 *threads*	10 s
Memória	`sysbench memory`, escrita seq., bloco 1 KiB	10 s
	idem, escrita sequencial, bloco 1 MiB	10 s
	idem, leitura sequencial, bloco 1 MiB	10 s
	idem, leitura aleatória num *buffer* de 256 MiB	10 s
Disco	`fio`, `libaio`, `direct=1`, ficheiro de 4 GiB: leitura e escrita aleatórias 4 KiB, $Q = 1$; mista 50/50 aleatória 4 KiB, $Q = 1$ e $Q = 32$; leitura e escrita sequenciais 1 MiB, $Q = 16$	3+15 s
Rede	`iperf3` TCP: convidado→anfitrião, inverso, 4 ligações	10 s
	`iperf3` UDP: 1 Mbit/s, 100 Mbit/s e 1 Gbit/s	10 s
	`ping`: 50 pacotes ICMP a 200 ms (RTT)	10 s

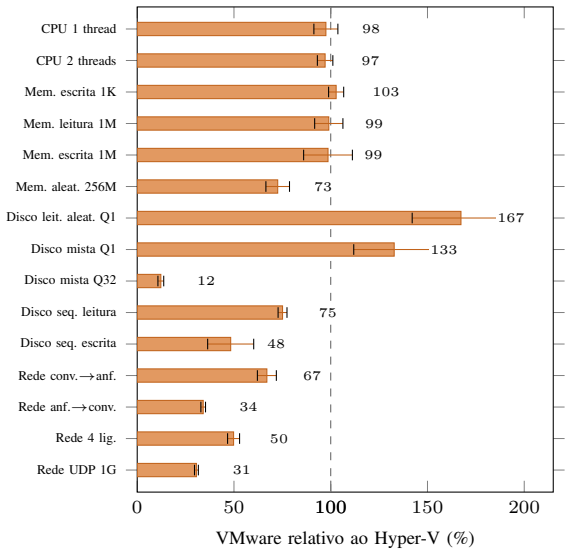


Figura 2. Débito do Tipo 2 relativo ao Tipo 1 (Hyper-V = 100%, tracejado). Barras de erro: $\sigma_{\mathrm{VMware}}/\mu_{\mathrm{Hyper\text{-}V}}$ (não incluem a dispersão do Hyper-V, indicada na Tabela IV).

da pilha de rede do anfitrião. Em todos os casos, $N = 10$ repetições, com aquecimento.

V. RESULTADOS EXPERIMENTAIS

A Tabela IV consolida as métricas principais, e a Fig. 2 sintetiza o desempenho relativo do Tipo 2. Todas as repetições foram mantidas, incluindo três valores atípicos no VMware (uma escrita sequencial de 266 MiB/s, uma corrida de CPU com 1 *thread* a 245 eventos/s e um RTT máximo de 10 ms), que aumentam o desvio-padrão mas não alteram a significância das conclusões.

A. Processador

Com um *thread*, o Hyper-V obteve 295,2 eventos/s e o VMware 287,9 ($-2{,}5\%$, $p = 0{,}241$). Com dois *threads*, os valores foram 552,8 e 536,5 ($-2{,}9\%$, $p = 0{,}055$). Nenhuma das diferenças é significativa, e o fator de escala de 1 para 2 *threads* é praticamente igual (1,87 no Hyper-V e 1,86 no VMware). O resultado está de acordo com a literatura: com

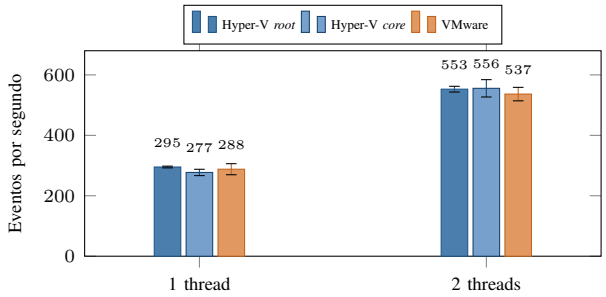


Figura 3. Processador ($\mu \pm \sigma$): paridade entre os três cenários com dois *threads*.

virtualização assistida por hardware, o código de utilizador executa diretamente no processador em ambas as arquiteturas [9], [13], e fica a menos de 5% do nativo com um *thread* (Tabela V). A diferença está na *estabilidade*: o coeficiente de variação foi de 1,0–1,7% no Hyper-V e de 4,2–6,4% no VMware. Isto é compatível com o facto de os vCPU do Tipo 2 serem *threads* de um processo comum, sujeitos à concorrência das restantes tarefas do Windows.

Efeito do escalonador. Repetiu-se o teste de processador com o *core scheduler* (`hypervisorschedulertype core`), o predefinido no Windows Server, mantendo a mesma topologia (Fig. 3). Com dois *threads*, o débito foi equivalente (555,6 contra 552,8 eventos/s, $+0{,}5\%$, $p = 0{,}77$). Com um *thread*, foi 6,1% inferior (277,2 contra 295,2, $p < 0{,}001$), com maior dispersão (percentil 95 de 4,56 contra 3,59 ms). Assim, para uma máquina com vCPU independentes, a escolha do escalonador não altera o débito agregado. A pequena perda com um único *thread* é compatível com o custo adicional de coordenação do isolamento por núcleo físico que o *core scheduler* impõe [11]. O escalonador predefinido do Windows 11 não favorece, portanto, o Hyper-V face ao do Windows Server.

B. Memória

Nos três testes sequenciais, as diferenças estão entre $-1{,}4\%$ e $+2{,}8\%$ e nenhuma é significativa: tanto a largura de banda da RAM (blocos de 1 MiB, que excedem as *caches*) como o acesso residente em *cache* L1 (blocos de 1 KiB) são equivalentes. O acesso *aleatório* num *buffer* de 256 MiB, porém, é 27,4% mais lento no VMware (441 contra 608 MiB/s, $p < 0{,}001$). É o único teste de memória que provoca falhas sistemáticas de TLB e, portanto, percursos completos pelas tabelas de páginas do convidado e do hipervisor. O seu custo depende do tamanho das páginas que o hipervisor usa para mapear a memória do convidado [10]. A diferença é compatível com o Hyper-V mapear a memória das partições com páginas grandes, enquanto no VMware a memória do convidado pertence ao processo `vmware-vmx` e é gerida pelo Windows, que por omissão a atribui em páginas de 4 KiB. Conclui-se que a memória não tem *overhead* relativo no acesso sequencial (embora ambos fiquem 12–20% abaixo do nativo), mas tem um custo mensurável no Tipo 2 para cargas com fraca localidade, como bases de dados ou estruturas de dados dispersas.

Tabela IV
RESULTADOS: HYPER-V (TIPO 1, ESCALONADOR *root*) VS. VMWARE WORKSTATION (TIPO 2), $\mu \pm \sigma$, $N = 10$

Subsistema	Métrica	Hyper-V	VMware	$\Delta\%$	p (Welch)	Melhor
CPU	Eventos/s, 1 *thread*	295,2 ± 2,9	287,9 ± 18,3	−2,5	0,241	n.s.
	Eventos/s, 2 *threads*	552,8 ± 9,5	536,5 ± 22,3	−2,9	0,055	n.s.
	Latência p95, 2 *threads* (ms)	4,44 ± 0,06	4,58 ± 0,25	+3,0	0,132	n.s.
Memória (MiB/s)	Escrita seq., bloco 1 KiB	4 226 ± 267	4 346 ± 163	+2,8	0,244	n.s.
	Leitura seq., bloco 1 MiB	29 140 ± 859	28 840 ± 2 122	−1,0	0,686	n.s.
	Escrita seq., bloco 1 MiB	22 499 ± 1 181	22 194 ± 2 835	−1,4	0,759	n.s.
	Leitura aleatória, 256 MiB	**608 ± 33**	**441 ± 37**	**−27,4**	**<0,001**	**Tipo 1**
Disco	Leitura aleatória 4K, $Q = 1$ (IOPS)	3 986 ± 183	6 668 ± 1 006	+67,3	<0,001	Tipo 2
	latência média / p99 (μs)	249 / 356	151 / 182	−39,5 / −49,0	<0,001 / <0,001	Tipo 2
	Escrita aleatória 4K, $Q = 1$ (IOPS)	5 393 ± 295	6 282 ± 770	+16,5	0,006	Tipo 2
	Mista 4K, $Q = 1$ (IOPS totais)	4 656 ± 206	6 183 ± 971	+32,8	<0,001	Tipo 2
	Mista 4K, $Q = 32$ (IOPS totais)	**75 399 ± 2 698**	**9 161 ± 1 152**	**−87,9**	**<0,001**	**Tipo 1**
	Leitura sequencial 1M, $Q = 16$ (MiB/s)	1 293 ± 31	972 ± 30	−24,9	<0,001	Tipo 1
	Escrita sequencial 1M, $Q = 16$ (MiB/s)	1 651 ± 232	798 ± 196	−51,7	<0,001	Tipo 1
Rede	TCP convidado→anfitrião (Mbit/s)	3 546 ± 241	2 374 ± 172	−33,0	<0,001	Tipo 1
	TCP anfitrião→convidado (Mbit/s)	6 165 ± 539	2 100 ± 73	−65,9	<0,001	Tipo 1
	TCP 4 ligações (Mbit/s)	4 584 ± 767	2 282 ± 143	−50,2	<0,001	Tipo 1
	RTT médio / máximo (ms)	0,46 / 0,78	0,51 / 2,33	+9,6 / +197	0,287 / 0,129	n.s.
	UDP 1 Mbit/s: *jitter* (ms)	0,044 ± 0,027	0,008 ± 0,005	−82,7	0,002	Tipo 2
	UDP 100 Mbit/s: *jitter* (ms)	0,020 ± 0,010	0,126 ± 0,039	+547	<0,001	Tipo 1
	UDP 1 Gbit/s: débito (Mbit/s) / perda (%)	977 / 2,3	299 / 0,0	−69,4 / –	<0,001 / 0,003	Tipo 1

n.s.: diferença não significativa ($p \geq 0{,}05$). Convenção de sinais de $\Delta\%$ conforme a Secção III-D.

C. Armazenamento

O armazenamento apresenta dois regimes opostos.

Fila unitária ($Q = 1$): vantagem do Tipo 2. Com um pedido de cada vez, o VMware obteve mais 67% de IOPS em leitura aleatória, com latência média de 151 μs contra 249 μs e um percentil 99 de 182 μs contra 356 μs. Este comportamento é compatível com a *cache* de ficheiros do Windows. O ficheiro de teste de 4 GiB, escrito pelo `fio` na preparação, cabe nos 16 GB do anfitrião, e o `vmware-vmx` acede ao `.vmdk` através do sistema de ficheiros. Assim, as leituras "diretas" do convidado são servidas a partir da RAM do anfitrião. A opção `direct=1` elimina a *cache* do convidado, mas não a do anfitrião. A escrita beneficia do mesmo mecanismo em modo *write-back* (+16,5% IOPS): o convidado recebe confirmação antes de os dados chegarem ao SSD, o que melhora a latência à custa da durabilidade em caso de falha de energia. No Hyper-V, cada pedido percorre o VMBus até ao NVMe, sem esta *cache*.

Paralelismo e débito: vantagem do Tipo 1. Com 32 pedidos em fila, o Hyper-V passou de 4 656 para 75 399 IOPS (16 vezes mais), com uma latência média (ponderada entre leituras e escritas) de apenas 424 μs. O VMware passou de 6 183 para 9 161 (1,5 vezes), com uma latência de 3 541 μs: uma diferença de -88% ($p < 0{,}001$). A lei de Little (3) reproduz ambos os valores: $32/424\,\mu\text{s} \approx 75\,500$ e $32/3\,541\,\mu\text{s} \approx 9\,036$ IOPS, a menos de 1,5% do medido. No VMware, a latência cresce na proporção da fila e o débito satura em cerca de 9 000 IOPS. O caminho de E/S está, portanto, *serializado*, como é esperado de um controlador emulado (LSI Logic) tratado por um processo em modo utilizador. O controlador sintético do Hyper-V, pelo contrário, despacha os pedidos em paralelo até ao NVMe. O mesmo limite reflete-se no débito sequencial com blocos de 1 MiB: -25% em leitura e -52% em escrita no Tipo 2 (Fig. 4).

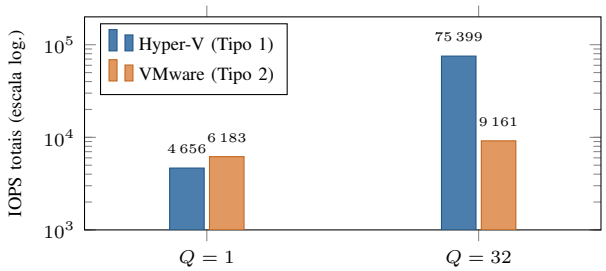


Figura 4. E/S aleatória mista 4 KiB: o Tipo 2 é mais rápido com $Q = 1$ (*cache* do anfitrião), mas não escala com a profundidade de fila.

D. Rede

O Hyper-V obteve débitos TCP 33% superiores no sentido convidado→anfitrião, 66% superiores no sentido inverso (neste sentido, o `iperf3` do anfitrião chegou a 87% de CPU, pelo que o valor do Hyper-V pode estar limitado pelo próprio gerador de tráfego) e 50% superiores com quatro ligações paralelas, todos com $p < 0{,}001$ (Fig. 5). A latência ida-e-volta média é equivalente (0,46 contra 0,51 ms, $p = 0{,}29$), mas o VMware apresenta picos até 2,3 ms e uma variação quatro vezes superior. A diferença resulta do tipo de dispositivo: a placa `e1000` do VMware é emulada, e cada acesso aos seus registos implica uma saída para o monitor e processamento no `vmware-vmx`, enquanto o `hv_netvsc` do Hyper-V troca pacotes em lote pelo VMBus. No envio a partir do convidado, o processo `iperf3` do anfitrião consumiu 31,6% de CPU no VMware, contra 9,4% no Hyper-V, para um débito inferior:

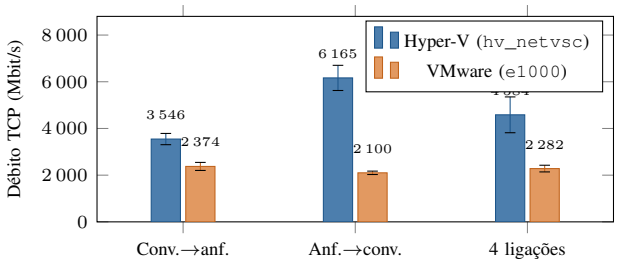


Figura 5. Débito de rede convidado↔anfitrião ($\mu \pm \sigma$).

cerca de 75 contra 376 Mbit/s por ponto percentual. Ou seja, no VMware a receção de cada bit custa mais CPU no anfitrião. Com a placa paravirtualizada `vmxnet3`, a diferença seria provavelmente menor. Não foi ensaiada para manter a configuração predefinida (Secção VI). Os ensaios UDP confirmam este padrão. Com tráfego mínimo (1 Mbit/s), o *jitter* é residual nos dois cenários (0,044 contra 0,008 ms). Com 100 Mbit/s, porém, o *jitter* do VMware é seis vezes superior (0,126 contra 0,020 ms, $p < 0{,}001$). A 1 Gbit/s, o Hyper-V entrega 977 Mbit/s com 2,3% de perda, enquanto o VMware não ultrapassa os 299 Mbit/s. Não perde pacotes, mas apenas porque o limite está no envio, ou seja, na própria placa emulada: o débito máximo de pacotes UDP do Tipo 2 é cerca de um terço do do Tipo 1.

À semelhança de Louro [3], usou-se o `iperf3` em TCP, para o débito, e em UDP, para o *jitter*. Nesse trabalho, porém, o tráfego atravessava a rede física entre os dois nós de um *cluster*, com uma janela TCP fixa de 20 KB: as máquinas sobre Hyper-V atingiram 207 a 222 Mbit/s e um *jitter* de 0,39 a 1,07 ms [3, Figs. 4.3–4.4]. Por medirem caminhos diferentes, os valores absolutos não são comparáveis.

E. Overhead Face ao Sistema Nativo

A Tabela V compara cada hipervisor com o sistema de referência, aplicando a Eq. (2) com o nativo no lugar do Hyper-V. No processador, o *overhead* é pequeno nos dois cenários: $-1{,}9\%$ (Hyper-V) e $-4{,}4\%$ (VMware) com um *thread*, e $-8{,}2\%$ e $-10{,}9\%$ com dois. O nativo escala de forma perfeita de 1 para 2 *threads* (fator 2,00), contra 1,87 e 1,86 nos convidados, e tem uma latência p95 de 3,30 ms, contra 4,44 e 4,58 ms. O custo da virtualização do processador está, portanto, sobretudo na execução simultânea dos dois vCPU, que competem com os *threads* do anfitrião, e não na execução de cada instrução. Cerca de 2 pontos percentuais deste valor explicam-se pela diferença de frequência (2,35 contra 2,40 GHz).

Na memória sequencial com blocos de 1 MiB, os dois hipervisores perdem 12–20% face ao nativo, sem diferença significativa entre eles. Este custo é, assim, comum às duas arquiteturas. No acesso aleatório, o *overhead* é de 26,8% no Hyper-V e de 46,9% no VMware. É o maior custo de todo o processador e da memória, o que confirma que a tradução de endereços em dois níveis é o principal custo da virtualização da memória e que é agravada no Tipo 2 (Secção V-B). O teste

Tabela V
OVERHEAD FACE AO SISTEMA NATIVO (μ, $N = 10$)

Métrica	Nativo	Hyper-V	VMware
Linux nativo, 2,4 GHz sem turbo (sysbench)			
CPU, 1 *thread* (eventos/s)	301,0	$-1{,}9\%$	$-4{,}4\%$
CPU, 2 *threads* (eventos/s)	602,1	$-8{,}2\%$	$-10{,}9\%$
CPU, latência p95, 2 *threads* (ms)	3,30	$+34{,}6\%$	$+38{,}7\%$
Mem. escrita seq. 1 KiB (MiB/s)	3 633	$+16{,}3\%$	$+19{,}6\%$
Mem. leitura seq. 1 MiB (MiB/s)	33 199	$-12{,}2\%$	$-13{,}1\%$
Mem. escrita seq. 1 MiB (MiB/s)	27 882	$-19{,}3\%$	$-20{,}4\%$
Mem. leitura aleat. 256 MiB (MiB/s)	**831**	$\mathbf{-26{,}8\%}$	$\mathbf{-46{,}9\%}$
Windows anfitrião, hipervisor desligado (fio, windowsaio)			
Leitura aleat. 4K, $Q = 1$ (IOPS)	7 758	$-48{,}6\%$	$-14{,}1\%$
latência média (μs)	127	$+95{,}5\%$	$+18{,}3\%$
Escrita aleat. 4K, $Q = 1$ (IOPS)	15 382	$-64{,}9\%$	$-59{,}2\%$
Mista 4K, $Q = 1$ (IOPS totais)	10 259	$-54{,}6\%$	$-39{,}7\%$
Mista 4K, $Q = 32$ (IOPS totais)	54 494	$+38{,}4\%$	$-83{,}2\%$
Leitura seq. 1M, $Q = 16$ (MiB/s)	1 218	$+6{,}2\%$	$-20{,}2\%$
Escrita seq. 1M, $Q = 16$ (MiB/s)	1 910	$-13{,}5\%$	$-58{,}2\%$
Windows anfitrião, iperf3 em loopback (indicativo)			
TCP envio/inverso (Mbit/s)	18 084/17 614	$-80/-65\%$	$-87/-88\%$
TCP 4 ligações (Mbit/s)	45 070	$-89{,}8\%$	$-94{,}9\%$

Todas as diferenças face ao nativo são significativas ($p < 0{,}05$, Welch). Desvios-padrão e IC95 no ficheiro de resultados [15].

com blocos de 1 KiB foi *mais rápido* nos convidados (+16 a +20%). Neste teste, cada bloco é um evento, e o resultado depende do custo fixo do `sysbench` por evento, que difere entre os ambientes. Não é, por isso, interpretado como ganho da virtualização.

No disco, a referência é o anfitrião Windows, com outra pilha de E/S (NTFS e `windowsaio`, em vez de `ext4` e `libaio`), pelo que a comparação é indicativa. Com $Q = 1$, os dois hipervisores perdem 14–65% de IOPS face ao anfitrião, e a latência média de leitura do Hyper-V é quase o dobro da nativa (249 contra 127 μs): é o custo, por pedido, de atravessar o VMBus e a partição raiz. O VMware perde menos em leitura (-14%) porque é servido em parte pela *cache* do anfitrião. Com $Q = 32$ e em leitura sequencial, o Hyper-V *excede* a referência ($+38\%$ e $+6\%$). Isto não significa que a virtualização acelere o disco, mas que o `libaio` do Linux sustenta mais pedidos simultâneos do que o `windowsaio` do `fio` no Windows. O VMware, pelo contrário, fica a 17% do anfitrião com $Q = 32$ e a 42% em escrita sequencial. O *loopback* do anfitrião atingiu 18 084 Mbit/s (TCP), 17 614 no sentido inverso e 45 070 com quatro ligações. Os convidados atingem 10–35% deste limite no Hyper-V e 5–13% no VMware. Como o *loopback* não atravessa nenhum dispositivo de rede, estes valores indicam apenas a margem que cada comutador virtual deixa por aproveitar.

VI. LIMITAÇÕES E AMEAÇAS À VALIDADE

- **Referência:** não existe um único sistema nativo equivalente aos dois convidados. O processador e a memória foram comparados com um Linux nativo arrancado por USB (*kernel* 7.0.0-30, contra 7.0.0-38 nos convidados), à frequência fixa de 2,4 GHz; o disco e a rede, com o Windows anfitrião (NTFS e `windowsaio`; *loopback*, porque o caminho convidado↔anfitrião não existe sem virtualização). O *overhead* face ao nativo no disco e

na rede inclui, por isso, as diferenças entre sistemas operativos, e não só o custo do hipervisor. A referência com *turbo* não foi usada, porque o anfitrião Windows não o ativou durante os ensaios.

- **Dispositivos predefinidos:** o VMware usou dispositivos emulados (LSI Logic, `e1000`) e o Hyper-V paravirtualizados. Isto reflete a configuração por omissão de cada produto, mas exagera a diferença face a um VMware com `vmxnet3` e PVSCSI/NVMe. Os escalonadores de E/S do convidado também diferiram (`none` e `mq-deadline`).
- ***Cache* do anfitrião no Tipo 2:** o ficheiro de teste de 4 GiB cabe na memória do anfitrião e o `direct=1` só elimina a *cache* do convidado. Assim, com $Q = 1$, o VMware é em parte medido a partir da RAM do anfitrião e o Hyper-V a partir do NVMe. Os dois caminhos não são equivalentes, e a vantagem do Tipo 2 com $Q = 1$ não deve ser lida como maior desempenho do disco virtual.
- **Assimetria de modo:** no cenário Hyper-V, o Windows executou como partição raiz com segurança baseada em virtualização ativa; no cenário VMware, sem hipervisor. Esta diferença é inerente às duas arquiteturas no mesmo equipamento.
- **Mecanismo da memória:** a explicação da diferença na leitura aleatória (tamanho das páginas) é inferida a partir do padrão dos resultados e da literatura, e não foi medida diretamente.
- **Equipamento:** um portátil com Windows 11 Home (edição em que o Hyper-V não é oficialmente suportado), em configuração de nó único e com 2 vCPU e 4 GB por máquina. Os resultados podem não se generalizar a servidores nem a infraestruturas com vários nós, em que a rede física também pesa.
- ***Benchmarks* sintéticos:** medem subsistemas isolados e não cargas reais, que os combinam.

VII. Conclusões

Este trabalho comparou experimentalmente um hipervisor nativo (Hyper-V) e um hipervisor *host-based* (VMware Workstation 17.6) no mesmo equipamento, com convidados Ubuntu Server 26.04 com os mesmos recursos e vCPU independentes. Nas condições ensaiadas:

1) **Processador e memória sequencial** não apresentam *overhead* relativo significativo: a virtualização assistida por hardware torna-os equivalentes. O Hyper-V foi, contudo, mais estável (coeficiente de variação inferior a 2%).
2) **O acesso aleatório à memória** é 27% mais lento no Tipo 2, consistente com um maior custo da tradução de endereços em dois níveis.
3) **No armazenamento**, o Tipo 2 é mais rápido com pedidos isolados, porque a *cache* do anfitrião serve as leituras e confirma as escritas antes de chegarem ao disco. Com concorrência, porém, o seu caminho de E/S satura em cerca de 9 000 IOPS, como a lei de Little confirma, e perde 88% de IOPS com $Q = 32$ e 25–52% de débito sequencial.
4) **Na rede**, o Tipo 1 é 33–66% mais rápido em TCP, suporta um débito UDP três vezes superior e tem um *jitter* seis vezes inferior sob carga moderada, devido aos dispositivos paravirtualizados.
5) **Face ao sistema nativo**, à mesma frequência, o processador perde 2–11% nos dois hipervisores, sobretudo na execução simultânea dos vCPU, e a memória sequencial 12–20%. O acesso aleatório à memória é o maior custo: 27% no Tipo 1 e 47% no Tipo 2.
6) **O escalonador do Hyper-V** (*root* ou *core*) não altera o débito de uma máquina com vCPU independentes.

Para cargas de centro de dados, com E/S concorrente, rede intensiva e acesso disperso à memória, os resultados favorecem claramente o hipervisor nativo. O Tipo 2 mantém-se adequado para estações de trabalho com cargas interativas e pouco paralelas. Num anfitrião Windows, é ainda indispensável confirmar que o hipervisor de Tipo 2 executa no seu modo nativo (`CPL0`) e não sobre o Hyper-V. Como trabalho futuro, propõe-se repetir os ensaios com dispositivos paravirtualizados no VMware e avaliar cargas reais, como bases de dados e servidores web.

Referências

[1] Microsoft, "Hyper-V overview," [Online]. Available: https://learn.microsoft.com/pt-pt/windows-server/virtualization/hyper-v/overview. [Accessed: Oct. 18, 2026].

[2] VMware, Inc., "VMware Workstation Pro documentation," [Online]. Available: https://www.vmware.com. [Accessed: Oct. 18, 2026].

[3] A. Louro, "Comparação do desempenho de infraestruturas virtualizadas de elevada disponibilidade usando hypervisors nativos ou containers," M.Sc. dissertation, Dept. Eng. Informática, Univ. Beira Interior, Covilhã, Portugal, 2018. [Online]. Available: https://ubibliorum.ubi.pt/handle/10400.6/9837.

[4] Universidade da Beira Interior, "Conteúdos das aulas teóricas da unidade curricular Virtualização e Centros de Dados," [Online]. Available: https://moodle.ubi.pt/. [Accessed: Oct. 18, 2026].

[5] A. Kopytov, "sysbench: Scriptable database and system performance benchmark," GitHub, 2020. [Online]. Available: https://github.com/akopytov/sysbench.

[6] J. Axboe, "Flexible I/O Tester (fio)," GitHub. [Online]. Available: https://github.com/axboe/fio. [Accessed: Oct. 18, 2026].

[7] G. J. Popek and R. P. Goldberg, "Formal requirements for virtualizable third generation architectures," *Commun. ACM*, vol. 17, no. 7, pp. 412–421, 1974.

[8] P. Barham *et al.*, "Xen and the art of virtualization," in *Proc. 19th ACM Symp. Operating Systems Principles (SOSP)*, 2003, pp. 164–177.

[9] K. Adams and O. Agesen, "A comparison of software and hardware techniques for x86 virtualization," in *Proc. 12th Int. Conf. ASPLOS*, 2006, pp. 2–13.

[10] R. Bhargava, B. Serebrin, F. Spadini, and S. Manne, "Accelerating two-dimensional page walks for virtualized systems," in *Proc. 13th Int. Conf. ASPLOS*, 2008, pp. 26–35.

[11] Microsoft, "Manage Hyper-V hypervisor scheduler types," [Online]. Available: https://learn.microsoft.com/windows-server/virtualization/hyper-v/manage/manage-hyper-v-scheduler-types. [Accessed: Oct. 22, 2026].

[12] Microsoft, "Windows Hypervisor Platform," [Online]. Available: https://learn.microsoft.com/virtualization/api/. [Accessed: Oct. 22, 2026].

[13] W. Felter, A. Ferreira, R. Rajamony, and J. Rubio, "An updated performance comparison of virtual machines and Linux containers," in *Proc. IEEE ISPASS*, 2015, pp. 171–172.

[14] J. D. C. Little, "A proof for the queuing formula: $L = \lambda W$," *Operations Research*, vol. 9, no. 3, pp. 383–387, 1961.

[15] H. Laia, A. Santos e D. Basílio, "Hyper-V vs. VMware: comandos, *scripts* e dados experimentais," GitHub, 2026. [Online]. Available: https://github.com/henriquelaia/vcd-hyperv-vs-vmware.